

THE CLAUDE BEAST

Claude Agent Teams

Multi-Agent Collaboration in Claude Code

February 2026 | Experimental Feature | Released with Opus 4.6

What Are Agent Teams?



Core Concept

Multiple Claude Code instances working together in parallel on the same project -- sharing tasks, communicating directly, and coordinating autonomously.

- One session acts as Team Lead (orchestrator), spawning independent Teammates
- Each teammate gets its own Claude Code instance with its own context window
- Teammates coordinate through a shared task list and direct messaging
- **Not just delegation** — teammates can discuss, challenge, and self-coordinate
- Evolves Claude Code from solo assistant to coordinated development squad

How to Enable Agent Teams

Option A: Environment Variable

```
export  
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

Option B: settings.json (Recommended)

```
{ "env": {  
  "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS":  
    "1"  
}  
}
```

Then simply ask Claude in natural language:

Prompt Example

```
"Create an agent team to explore this from different angles:  
one teammate on UX, one on technical architecture,  
one playing devil's advocate."
```

Claude handles team creation, teammate spawning, and coordination automatically.

Architecture: Four Core Components



Team Lead

The main Claude Code session.
Creates team, spawns teammates,
coordinates all work.



Teammates

Separate Claude instances.
Each with own context window.
Work independently on tasks.



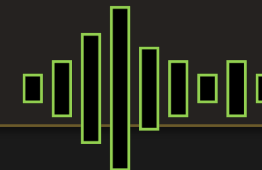
Task List

Shared work items on disk.
Dependency tracking built in.
Claim, execute, complete.



Mailbox

Direct messaging system.
Agent-to-agent communication.
Broadcasts + targeted msgs.



All state stored as JSON files on disk: `~/.claude/teams/` and `~/.claude/tasks/`

The 7 Primitives Under the Hood

SETUP

TeamCreate

Creates team directory
& config file

TaskCreate

Defines a unit of
work as JSON

Task (spawn)

Spawns a teammate
instance

EXECUTION

TaskList

Returns all tasks
with current status

TaskUpdate

Claims tasks & marks
them complete

COMMUNICATION

SendMessage

Messages between
any two agents

TeamDelete

Removes team config
& task files

Agent Types & Communication Model

BUILT-IN AGENT TYPES

General-purpose

Full tools, complex multi-step tasks

Inherits model

Explore

Read-only codebase discovery

Haiku (fast/cheap)

Plan

Research & context gathering

Inherits model

Bash

Terminal commands in separate context

Inherits model

PEER-TO-PEER COMMUNICATION

- Direct messages between any two agents
- Broadcast to all teammates (use sparingly)
- Shutdown request / response protocol
- Plan approval workflow for quality gates
- Idle notifications when agents finish
- **No polling needed** — auto-delivered



Key Insight

This is peer-to-peer, not hub-and-spoke. The API teammate can tell the frontend teammate directly that types are ready.

High-Impact Use Cases



Parallel Code Review

1

3 reviewers apply different lenses (security, performance, tests) to the same PR. Consistently surfaces issues single-pass reviews miss.



Competing Hypotheses

2

5 agents investigate different debugging theories and actively disprove each other. Eliminates anchoring bias from sequential investigation.



Cross-Layer Development

3

API, frontend, and test teammates each own their layer. Agents message each other directly when interfaces are ready.



QA Swarm

4

5 QA agents test pages, posts, links, SEO, and accessibility simultaneously. Real-world example: 10 issues found and prioritized in ~3 minutes.

Cost Considerations & Optimization

TOKEN USAGE SCALES LINEARLY

Solo session	~200K tokens	1x
3 subagents	~440K tokens	~2.2x
3 agent team	~800K tokens	~4x
5 agent team	~1.2M+ tokens	~6x+

COST OPTIMIZATION STRATEGIES

- Opus for the lead, Sonnet for teammates
- Start with 3-5 teammates (diminishing returns beyond)
- Plan first (~10K tokens), then parallelize
- Use targeted messages, avoid broadcasts
- Size tasks at 5-6 per teammate



When Teams Are Worth The Cost

- Research, review, and new features: extra tokens usually worthwhile
- Routine / sequential tasks: single session more cost-effective
- Rule of thumb: use teams when coordination benefit justifies 3-6x token cost

Key Takeaways & Getting Started

KEY TAKEAWAYS

- Agent Teams transform Claude Code into a coordinated development squad
- Architecture is simple: lead + teammates + tasks + messaging (all JSON on disk)
- Peer-to-peer communication is the differentiator vs. subagents
- Still experimental: no resume, one team per session, no nested teams
- Engineering management skills transfer directly to agent orchestration

BEST PRACTICES

- Start with review & research tasks (clear boundaries, low risk)
- Plan first, parallelize second (cheap planning, expensive execution)
- 3-5 teammates is the sweet spot for most workflows
- Use delegate mode (Shift+Tab) to keep lead focused on coordination
- Give each teammate clear file ownership to avoid conflicts



The Delegation Progression

Level 0: Solo Session → Level 1: Subagents (delegate + report) → Level 2: Agent Teams (coordinate independently)